

Chapter 10 Programming Languages

After studying this chapter, the student should be able to:

- Describe the evolution of programming languages from machine language to high-level languages.
- Understand how a program in a high-level language is translated into machine language.
- Distinguish between four computer language paradigm.
- Understand the procedural paradigm and the interaction between a program unit and data items in the paradigm.
- Understand the object-oriented paradigm and the interaction between a program unit and data items in the paradigm.
- Define functional paradigm and understand its applications.
- Define a declaration paradigm and understand its applications.
- Define common concepts in procedural and object-oriented languages.

10.1 EVOLUTION

- To write a program for a computer, we must use a computer language.
- A computer language is a set of predefined words that are combined into a program according to predefined rules (*syntax*).
- Over the years, computer languages have evolved from *machine language* to *high-level languages*.
 - Machine languages
 - Assembly languages
 - High level languages

10.1.1 Machine languages

- In the earliest days of computers, the only programming languages available were **machine languages**.
- Each computer had its own machine language, which was made of streams of 0s and 1s.
- In Chapter 5 we showed that in a primitive hypothetical computer, we need to use eleven lines of code to read two integers, add them, and print the result. These lines of code, when written in machine language, make eleven lines of binary code, each of 16 bits, as shown in Table 10.1.

Table 10.1 Code in machine language to add two integers

<i>Hexadecimal</i>	<i>Code in machine language</i>			
$(1FEF)_{16}$	0001	1111	1110	1111
$(240F)_{16}$	0010	0100	0000	1111
$(1FEF)_{16}$	0001	1111	1110	1111
$(241F)_{16}$	0010	0100	0001	1111
$(1040)_{16}$	0001	0000	0100	0000
$(1141)_{16}$	0001	0001	0100	0001
$(3201)_{16}$	0011	0010	0000	0001
$(2422)_{16}$	0010	0100	0010	0010
$(1F42)_{16}$	0001	1111	0100	0010
$(2FFF)_{16}$	0010	1111	1111	1111
$(0000)_{16}$	0000	0000	0000	0000

The only language understood by a computer is machine language.

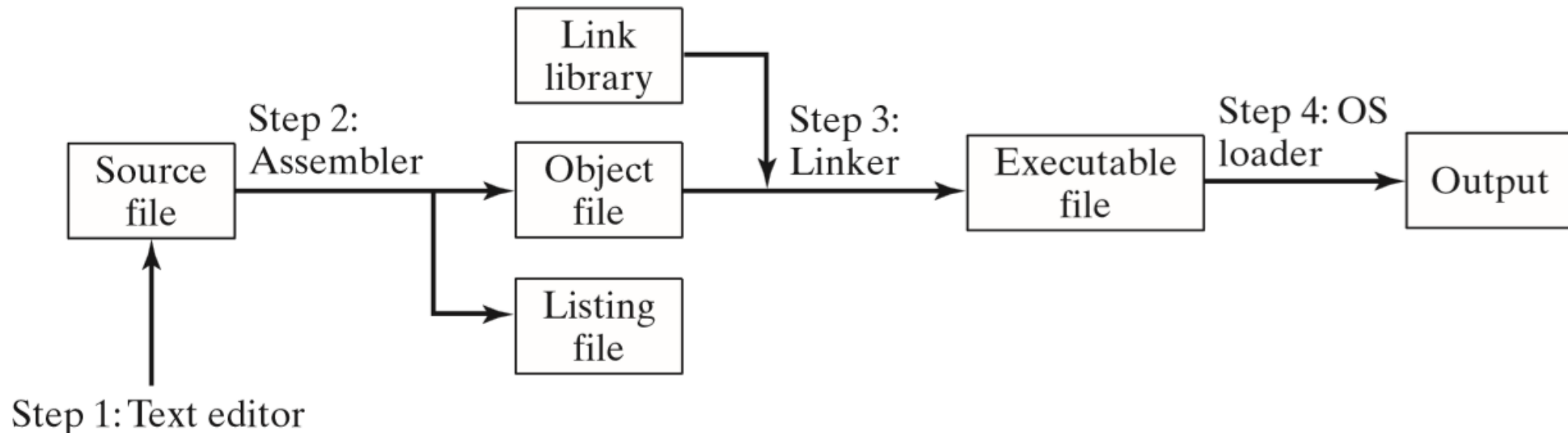
10.1.2 Assembly languages

- The next evolution in programming came with the idea of replacing binary code for instruction and addresses with symbols or mnemonics. Because they used symbols, these languages were first known as symbolic languages.
- The set of these mnemonic languages were later referred to as assembly languages.
- The assembly language for our hypothetical computer to replace the machine language in Table 10.2 is shown in Program 10.1.

Table 10.2 Code in assembly language to add two integers

Code in assembly language				Description
LOAD	RF	Keyboard		Load from keyboard controller to register F
STORE	Number1	RF		Store register F into Number1
LOAD	RF	Keyboard		Load from keyboard controller to register F
STORE	Number2	RF		Store register F into Number2
LOAD	R0	Number1		Load Number1 into register 0
LOAD	R1	Number2		Load Number2 into register 1
ADDI	R2	R0	R1	Add registers 0 and 1 with result in register 2
STORE	Result	R2		Store register 2 into Result
LOAD	RF	Result		Load Result into register F
STORE	Monitor	RF		Store register F into monitor controller
HALT				Stop

Assembly link execution cycle



1. A programmer uses a text editor to create a source file **.asm**
2. The assembler reads the source file and produces an object. Optionally, it produces a listing file **.lst**.
3. The linker reads the object file and the required libraries or functions from the link library, combines them with the object file and produces an executable file **.com**
4. The operating system loader utility reads the executable file into memory and branches the CPU to the program's starting address and the program begins to execute

10.1.3 High-level languages

- Although assembly languages greatly improved programming efficiency, they still required programmers to concentrate on the hardware they were using.
- Working with symbolic languages was also very tedious, because each machine instruction had to be individually coded.
- The desire to improve programmer efficiency and to change the focus from the computer to the problem being solved led to the development of high-level languages.
- Over the years, various languages, most notably BASIC, COBOL, Pascal, Ada, C, C++, and Java, were developed.
- Program 10.1 shows the code for adding two integers as it would appear in the C++ language.

```
#include <iostream>
using namespace std;

/*
This program reads two integers
and print their sum.
*/
int main()
{
    //local declaration
    int number1;
    int number2;
    int sum;

    //statements
    cin >> number1;
    cin >> number2;

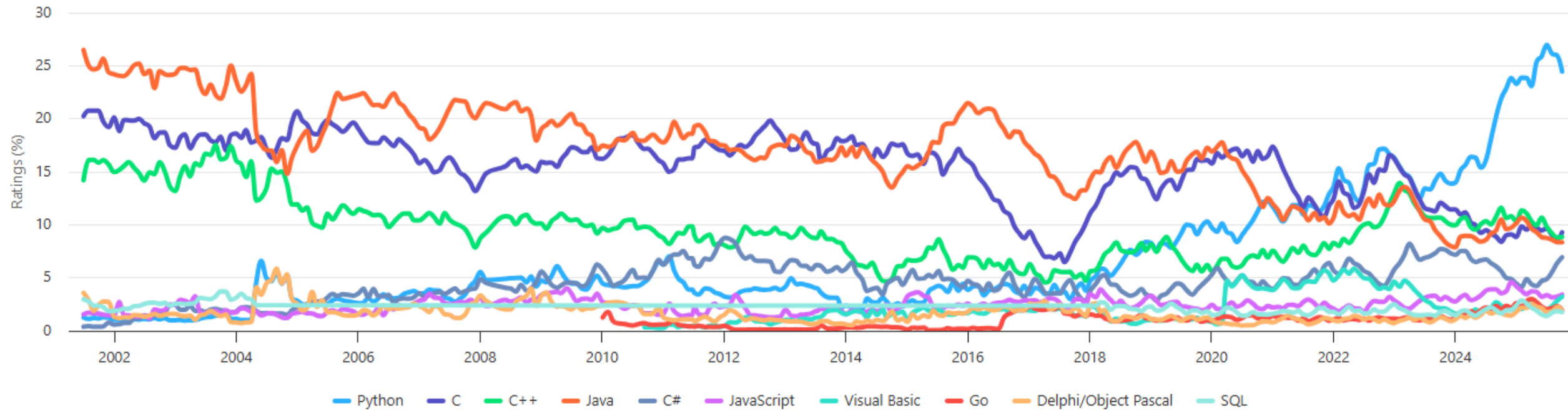
    sum = number1 + number2;
    cout << sum;

    return 0;
}
```

TIOBE Index for October 2025 - an indicator of the popularity of programming languages

TIOBE Programming Community Index

Source: www.tiobe.com



10.2 TRANSLATION

- Programs today are normally written in one of the high-level languages.
- To run the program on a computer, the program needs to be translated into the machine language of the computer on which it will run.
- The program in a high-level language is called the source program.
- The translated program in machine language is called the object program.
- Two methods are used for translation: ***compilation*** and ***interpretation***.

10.2.1 Compilation

- A ***compiler*** normally translates the whole source program into the object program.

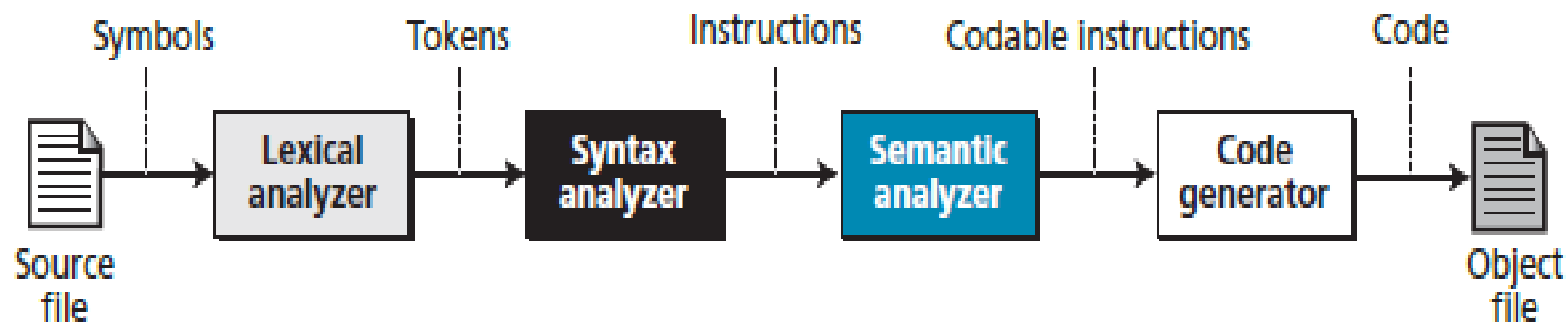
10.2.2 Interpretation

- Some computer languages use an ***interpreter*** to translate the source program into the object program.
- Interpretation refers to the process of translating each line of the source program into the corresponding line of the object program and executing the line.
- However, we need to be aware of two trends in interpretation: that used by some languages before Java and the interpretation used by Java.

10.2.3 Translation process

- **Compilation** and **interpretation** differ in that the first translates the whole source code before executing it, while the second translates and executes the source code a line at a time.
- Both methods, however, follow the same translation process shown in Figure 10.1.

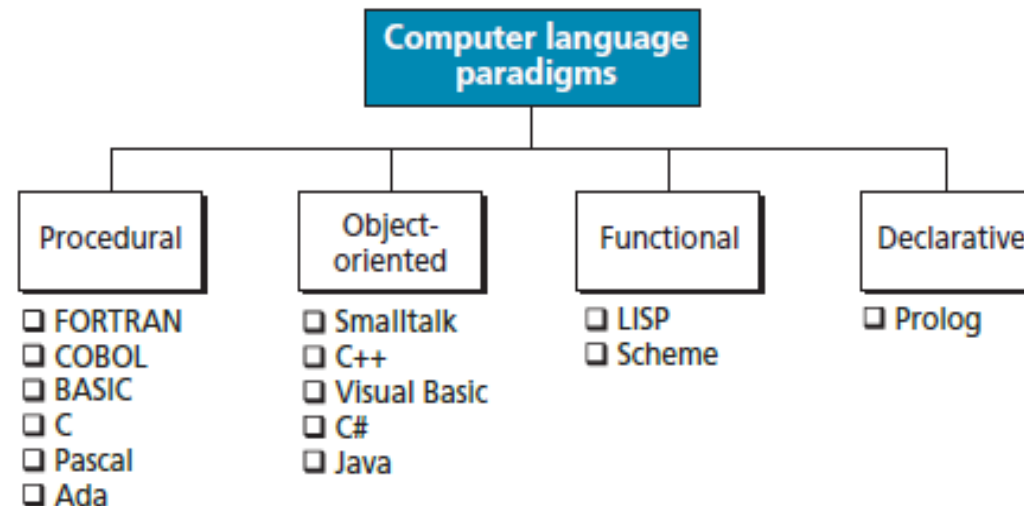
Figure 10.1 Source code translation process



10.3 PROGRAMMING PARADIGMS

- Today computer languages are categorized according to the approach they use to solve a problem.
- A paradigm, therefore, is a way in which a computer language looks at the problem to be solved.
- We divide computer languages into four paradigms:
 - *procedural*,
 - *object-oriented*,
 - *functional*, and
 - *declarative*.
- Figure 10.2 summarizes these.

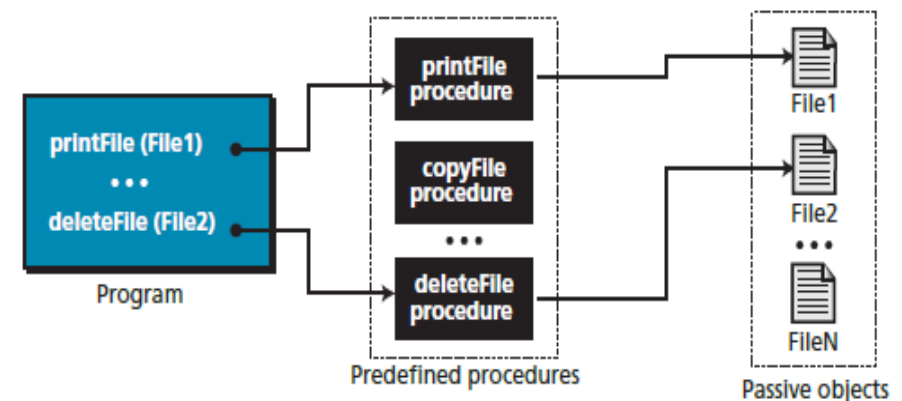
Figure 10.2 Categories of programming languages



10.3.1 The procedural paradigm

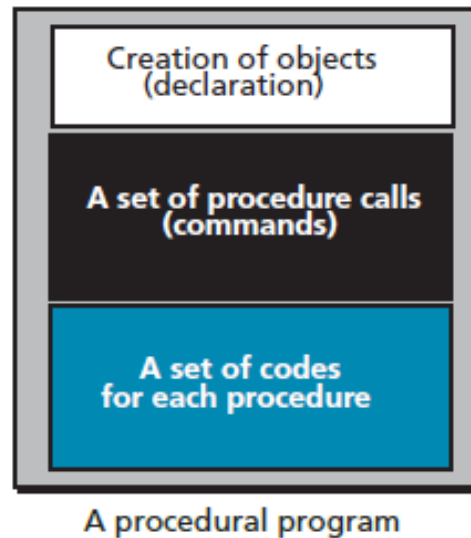
- In the procedural paradigm (or imperative paradigm) we can think of a program as an active agent that manipulates passive objects. We encounter many passive objects in our daily life: a stone, a book, a lamp, and so on. A passive object cannot initiate an action by itself, but it can receive actions from active agents.
- A program in a procedural paradigm is an active agent that uses passive objects that we refer to as data or data items.
- To manipulate a piece of data, the active agent (program) issues an action, referred to as a procedure. For example, think of a program that prints the contents of a file. The file is a passive object. To print the file, the program uses a procedure, which we call print.

Figure 10.3 The concept of the procedural paradigm



- A program in this paradigm is made up of three parts: ***a part for object creation, a set of procedure calls, and a set of code for each procedure.***
- Some procedures have already been defined in the language itself. By combining this code, the programmer can create new procedures.

Figure 10.4 The components of a procedural program



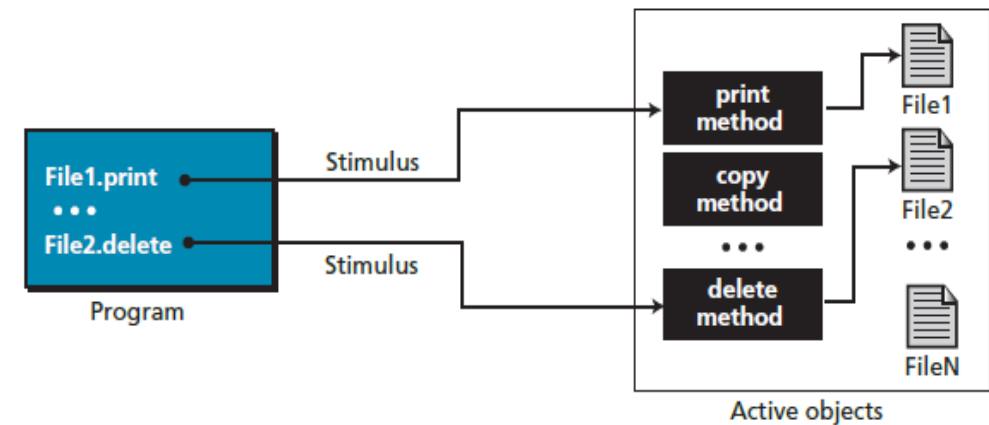
Procedural languages

- FORTRAN (FORmula TRANslation)
- COBO (COmmon Business-Oriented Language)
- Pascal
- C
- Ada

10.3.2 The object-oriented paradigm

- The object-oriented paradigm deals with active objects instead of passive objects. We encounter many active objects in our daily life: a vehicle, an automatic door, a dishwasher, and so on. The action to be performed on these objects are included in the object: the objects need only to receive the appropriate stimulus from outside to perform one of the actions.
- A file in an object-oriented paradigm can be packed with all the procedures—called methods in the object-oriented paradigm—to be performed by the file: printing, copying, deleting, and so on.
- The program in this paradigm just sends the corresponding request to the object.

Figure 10.5 The concept of an object-oriented paradigm

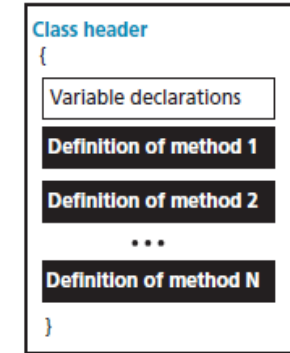


Object-oriented languages: C++, Java

Classes

- As Figure 10.5 shows, objects of the same type (files, for example) need a set of methods that show how an object of this type reacts to stimuli from outside the object's "territories".
- To create these methods, a unit called a **class** is used.

Figure 10.6 The components of a class



Methods

- In general, the format of methods are very similar to the functions used in some procedural languages.
- Each method has its header, its local variables, and its statement. This means that most of the features we discussed for procedural languages are also applied to methods written for an object-oriented program. In other words, we can claim that object-oriented languages are actually an extension of procedural languages with some new ideas and some new features.
- The C++ language, for example, is an object-oriented extension of the C language.

Inheritance

- In the object-oriented paradigm, as in nature, an object can inherit from another object. This concept is called inheritance.
- When a general class is defined, we can define a more specific class that inherits some of the characteristics of the general class, but also has some new characteristics.
- For example, when an object of the type Geometrical Shapes is defined, we can define a class called Rectangles. Rectangles are geometrical shapes with additional characteristics.

Polymorphism

- **Polymorphism** means “many forms”. Polymorphism in the object-oriented paradigm means that we can define several operations with the same name that can do different things in related classes.
- For example, assume that we define two classes, Rectangles and Circles, both inherited from the class Geometrical Shapes. We define two operations both named area, one in Rectangles and one in Circles, that calculate the area of a rectangle or a circle. The two operations have the same name.

10.3.3 The functional paradigm

- In the functional paradigm a program is considered a mathematical function. In this context, a function is a black box that maps a list of inputs to a list of outputs.

Figure 10.7 A function in a functional language

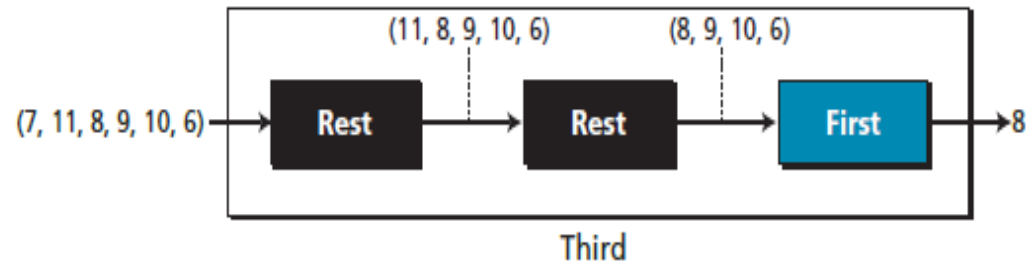


Functional languages:

- LISP
- Scheme

- First()** extracts the first element of a list.
- Rest()** extracts all the elements except the first.
- A program can define a function that extracts the third element of a list by combining these two functions as shown in Figure 10.8.

Figure 10.8 Extracting the third element of a list



10.3.4 The declarative paradigm

- A declarative paradigm uses the principle of logical reasoning to answer queries. It is based on formal logic defined by Greek mathematicians and later developed into first-order predicate calculus.
- Logical reasoning is based on deduction. Some statements (facts) are given that are assumed to be true, and the logician uses solid rules of logical reasoning to deduce new statements (facts).
- For example, the famous rule of deduction in logic is:

If(A is B) and (B is C), then (A is C)

Using this rule and the two following facts,

Fact 1: Socrates is a human $\rightarrow$ A is B

Fact 2: A human is mortal $\rightarrow$ B is C

we can deduce a new fact:

Fact 3: Socrates is mortal $\rightarrow$ A is C

Prolog

- One of the famous declarative languages is Prolog (PROgramming in LOGic), developed by A. Colmerauer in France in 1972.
- A program in Prolog is made up of facts and rules.
- For example, the previous facts about human beings can be stated as:

human(John)

mortal(human)

The user can then ask:

?-mortal(John)

and the program will respond with *yes*.

10.4 COMMON CONCEPTS

- Identifiers 識別字
- Data types 資料型別
- Variables 變數
- Literals 字面常數
- Constants 常數
- Inputs and Outputs 輸入/輸出
- Expressions 表示式
- Statements 敘述
- Subprograms (functions) 副程式/函數
- Return values 回傳值

Exercises

- P273 Problems
 - p10-4, p10-5, p10-6, p10-7, p10-23, p10-24, p10-25